

Code review — recapitulare-initiere-cpp

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 2 — 2026-10-01 — sha fdda59a

Runda 1 n-a ajuns la tine până acum — PDF-ul a rămas la mine, necomis. De asta o găsești mai jos întreagă, nu condensată. Nimic din ea nu e „restanță” în sensul rău: n-ai avut ce să repara. Dar o citești acum, fiindcă jumătate din ce urmează e aceeași regulă a doua oară.

Ai scris 156 de linii noi în `parcurgerea-vectoriilor.h` și ai rescris `main()`. Am compilat și am **rulat fiecare funcție nouă**. Nimic de mai jos nu e dedus din citit.

Poarta de compilare:  **proiectul NU se compilează**. 7 erori. Vezi B8.

Critice

B8 — Proiectul nu se compilează: 7 erori, în 5 locuri

`parcurgerea-vectoriilor.h:72` · `:91` · `:94` · `:166` · `main.cpp:20`

CUM E ACUM: `g++ -std=c++17 -fsyntax-only main.cpp` spune:

```
pv.h:78:1: error: function definition is not allowed here
pv.h:103:1: error: function definition is not allowed here
pv.h:147:1: error: function definition is not allowed here
pv.h:166:22: error: unknown type name 'intc'
pv.h:167:1: error: function definition is not allowed here
main.cpp:6:1: error: function definition is not allowed here
main.cpp:27:2: error: expected '}'

note: to match this '{' -- pv.h:66:13: void soll2(){
```

DE CE: cinci dintre cele șapte erori au **o singură cauză**. `soll2()` se deschide la `:66` și nu se închide niciodată. Ultima linie din corpul ei e `cout << contorPrimeVector(v, 5);` la `:71`, iar imediat după începe `//TEMA` și funcțiile noi.

Compilatorul nu ghicește unde voiai să se termine o funcție — merge înainte până găsește acolada potrivită. Deci citește `punctul_c`, `punctul_g`, `CifraMinima`, `areCifra` și chiar `main()` ca fiind **scrise înăuntrul lui `soll2`**. În C++ nu poți defini o funcție în interiorul alteia, și de aceea primești de cinci ori același mesaj: `function definition is not allowed here`. Iar `expected '}'` de la sfârșitul lui `main.cpp` e compilatorul care ajunge la capătul fișierului tot căutând acolada lui `soll2`.

Astea cinci nu sunt cinci greșeli. Sunt **o acoladă lipsă**, raportată de cinci ori. Citește întotdeauna **prima** eroare și `note:`-ul de la coadă — el îți spune exact care `{` a rămas deschisă.

Celelalte două erori sunt reale și independente:

- `:166` — `bool areCifra(int n, intc)`. Lipsește spațiul: `int c`. Fără el, `intc` e citit ca **nume de tip** și nu există niciun tip cu numele ăsta.
- `:94` — `nou = nou * 10 + c[j];`. Punctul și virgula a intrat în paranteza pătrată: `c[j];`.
- `:91` — `r = 0;`. Nu există nicio variabilă `r`. Din context voiai `nou = 0;` (vezi B11).

Și încă trei, pe care nici nu le-ai văzut: după ce am reparat eu cele cinci de sus, au ieșit la suprafață erorile din `main.cpp`:

```
main.cpp:20:18: error: use of undeclared identifier 'cifminn'
main.cpp:21:13: error: use of undeclared identifier 'cifminn'
main.cpp:24:13: error: use of undeclared identifier 'cifminn'
```

La `:16` ai declarat `int cifmin = 9;` — cu doi `n` la capăt o folosești, `cifminn`. Compilatorul nu ajunsese niciodată aici, fiindcă se oprirea la acolada din header.

Dovada că ai comis după ce ai spart buildul: `bin/Debug/recapitulare.exe` din commit-ul `fdda59a` are 1 058 821 de octeți, diferit de cel din `2734bb4`. Deci ai avut un build reușit în timpul sesiunii. Dar `.exe`-ul comis **nu poate fi reconstruit din sursa comisă** — sursa nu compilează. Ai mai editat după ultimul F9 reușit, și ai comis fără să mai apeși F9.

FIX: pune `}` după `:71`, și apoi **apasă F9 înainte de fiecare commit**. Zece secunde, și prinde toate cele șapte.

B9 — `contorPrimeVector` numără indicii, nu elementele vectorului

`parcurea-vectoriilor.h:55` · `:57`

CUM E ACUM: rulat, după ce am reparat erorile de sintaxă:

```
contorPrimeVector({23, 5, 4, 3, 7}, 5) -> 3      trebuia 4
contorPrimeVector({10, 12, 14}, 3)      -> 2      trebuia 0
```

Al doilea exemplu e cel care spune tot: în `{10, 12, 14}` nu există niciun număr prim, și funcția răspunde `2`.

DE CE: în corpul buclei scrie `if(isPrim(i) == true)`. `i` e **contorul**, nu valoarea. Funcția ta se uită la `0, 1, 2, 3, 4, 5` — adică la indici — și numără câți dintre ei sunt primi: `2, 3, 5`, deci trei. Pe `{10, 12, 14}` se uită la `0, 1, 2, 3` și găsește `2` și `3`.

Semnul care îți dă diagnosticul fără să rulezi nimic: **parametrul `v` nu apare nicăieri în corpul funcției**. O funcție care primește un vector și nu-l citește niciodată nu poate răspunde nimic despre el. Răspunsul `3` pe primul exemplu e o coincidență — în `{23, 5, 4, 3, 7}` sunt chiar 4 primi, și el a nimerit 3.

Al doilea bug, în aceeași buclă: `i <= dim`. Cu `dim = 5`, `i` ajunge la `5`, iar elementele sunt pe `0, 1, 2, 3, 4`. Un element peste. Acum nu se vede, fiindcă nu atingi `v` deloc — dar în clipa în care repara primul bug, `v[5]` e memorie care nu-ți aparține.

FIX: `isPrim(v[i])` și `i < dim`.

B10 — `punctul_c` citește de la indicele 1, dar `main` umple de la 0

parcurgerea-vectoriilor.h:81 · main.cpp:11

CUM E ACUM: rulat pe un vector umplut ca în `main.cpp`, de la `v[0]`:

```
punctul_c({121, 45, 1221}, 3) -> 1221
                                trebuia "121 1221"
punctul_c({121}, 1)           -> (nu tipareste nimic)
                                trebuia 121
```

DE CE: bucla ta e `for(i = 1; i <= n; i++)` și citește `v[i]`. Deci atinge `v[1] ... v[n]`. Iar `main.cpp:11` umple `v[0] ... v[n-1]`:

```
for(int i = 0; i <n; i++)
{
    cin >> v[i];
}
```

Cele două convenții sunt decalate cu o poziție. Rezultatul: **primul element citit nu e niciodată procesat**, iar funcția citește în schimb `v[n]` — o poziție pe care nimeni n-a scris nimic. Pe exemplul cu un singur element, funcția nu se uită deloc la el: `v[1]` e gol, `v[0] = 121` rămâne neatins.

Ambele convenții sunt valide în sine. În manualele de bac vectorul se numerează de la 1 fiindcă așa e scris enunțul; în C++ indicele natural e 0. Ce **nu** e valid e să le amesteci în același program: una citește, alta procesează, și se ratează la mijloc. **Alege una și ține-o în tot fișierul.** Recomandarea mea: 0, fiindcă e cea pe care ți-o impune limbajul oriunde altundeva (`v[0]` e prima poziție reală).

Același decalaj e și în `punctul_g:106`.

FIX: `for(i = 0; i < n; i++)` în `punctul_c` și `punctul_g`, și digits pe `c[0]`.

B11 — Testul de palindrom verifică doar capetele, și tipărește oglinditul

parcurgerea-vectoriilor.h:89 · :92

CUM E ACUM: rulat, cu vectorul umplut de la 1 ca să treacă peste B10:

```
punctul_c({1231, 1551}, 2) -> "1321" "1551"
```

`1231` **nu e palindrom**, și totuși a trecut. Mai mult: a fost tipărit `1321`, nu `1231`.

DE CE: sunt două lucruri, și al doilea e ascuns de primul.

`if(c[1] == c[nr])` compară prima cifră extrasă cu ultima extrasă — adică unitățile cu prima cifră din stânga. Pentru `1231` asta e `1 == 1`, adevărat. Dar un palindrom cere ca **toate** perechile simetrice să fie egale: `c[1]==c[nr]`, `c[2]==c[nr-1]`, și așa mai departe până la mijloc. Tu verifici doar prima pereche. Orice număr care începe și se termină cu aceeași cifră trece: `1231`, `1991`, `12341`.

Al doilea: `nou = nou * 10 + c[j]` cu `j` de la `1` la `nr`. În `c[1]` ai pus **unitățile** (prima cifră scoasă cu `% 10`), deci construind de la `c[1]` înainte pui unitățile pe poziția cea mai semnificativă. Rezultatul e numărul **oglintit**. Pentru un palindrom adevărat oglinditul e identic cu originalul, așa că bug-ul ăsta e invizibil — până dai peste un fals pozitiv ca `1231`, și vezi `1321`.

Al treilea, care în codul tău e eroare de compilare: `nou` e declarat la `:79`, în afara buclei, și `r = 0;` de la `:91` era încercarea ta de a-l reseta. Cu `r` corectat în `nou`, merge. Fără reset, cifrele numerelor consecutive se lipesc:

```
punctul_c({121, 1221}, 2), fara reset -> 1211211221
```

FIX: parcurge perechile simetrice (`c[j] == c[nr+1-j]` pentru `j` până la `nr/2`), tipărește `v[i]` direct — ai deja numărul, nu trebuie reconstruit — și pune `nou = 0` la începutul fiecărei iterații.

B12 — `punctul_g` se blochează la infinit

```
parcurea-vectoriilor.h:109 · :117
```

CUM E ACUM: am pus un `alarm(3)` și am rulat:

```
punctul_g({38, 47}, 2) -> omorata dupa 3 secunde, fara nicio ieșire
```

DE CE: bucla e `while(s > 9)`, iar singura atingere a lui `s` în corp e `s = s + c[j]` la `:117`. Un `+=` nu poate face un număr mai mic. `s` pornește de la `38` și **crește** la fiecare tur: 38, 46, 52, 54, 58... Condiția `s > 9` a fost adevărată la primul tur și va rămâne adevărată la infinit.

Asta e fix aceeași greșală ca B3 de la runda 1 (`celMaiMareNumar`, `celMaiMareNumar2`) — a treia buclă `while` din repo care nu se poate termina. Regula, în cuvinte simple: **variabila din condiție**

trebuie să se miște în direcția care face condiția falsă. La `while(n > 0)` ai nevoie de `n = n / 10`; la `while(s > 9)` ai nevoie ca `s` să **scadă**.

Ce voiai de fapt: rădăcina digitală — înlocuiești `s` cu **suma cifrelor lui**, repetat, până rămâne o cifră. Deci `s` trebuie să devină suma, nu să adune suma la el însuși. Și adunarea trebuie să fie **înăuntru** `while`-ului: tu ai calculat cifrele în `c` înăuntru, dar suma lor o faci la `:120–124`, după ce bucla s-a încheiat.

Mai e și `c[j]` la `:117` cu `j` nicăieri inițializat — citești o poziție din `c` aleasă la întâmplare. Nu schimbă concluzia (bucloa n-ar termina nici cu `j = 1`), dar e a doua problemă în aceeași linie.

FIX: înăuntru `while`-ului, după ce ai cifrele în `c`, calculează suma într-o variabilă nouă și **atribuie-o** lui `s`:

```
while(s > 9)
{
    nr = 0;
    for(x = s; x > 0; x = x / 10)
    {
        nr++;
        c[nr] = x % 10;
    }
    int suma = 0;
    for(j = 1; j <= nr; j++)
    {
        suma = suma + c[j];
    }
    s = suma;
}
```

B13 — În `punctul_g`, jumătatea a doua e în afara buclei

`parcurgerea-vectoriilor.h:125` · `:128` · `:131`

CUM E ACUM: acolada de la `:125` închide `for(i = 1; i <= n; i++)`. Blocul `if(s % 2 == 0)` de la `:128` e **după** ea. Deci nu se execută o dată pentru fiecare element, ci o singură dată la final — și folosește `v[i]` la `:131`, unde `i` are deja valoarea `n + 1`, fiindcă așa a ieșit din buclă.

DE CE: În C++ indentarea nu înseamnă nimic pentru compilator. Blocul unei bucle e delimitat de acolade, și acoladele tale spun altceva decât spune indentarea. La `:125` închizi for-ul, la `:128` continui ca dacă ai fi încă înăuntru.

Consecința e dublă. Întâi: `s` are, la `:128`, doar valoarea rămasă de la **ultimul** element — testul de paritate se face o singură dată, pentru ultimul număr din vector, nu pentru fiecare. Apoi: `i` e `n + 1`, deci `v[i]` e la **două poziții** după ultimul element real (una de la convenția 1-based din B10, una de la ieșirea din buclă). Citești ce s-a nimerit în memorie.

Semnul care îl prinde fără compilator: o variabilă de buclă (`i`) folosită în afara buclei care a declarat-o are sens doar dacă ai **nevoie** de valoarea ei finală. Aici n-ai; ai nevoie de elementul curent, și elementul curent există doar înăuntru.

FIX: mută `}` de la `:125` până după `:141`, ca tot blocul `if` să intre în `for`.

B14 — „Funcție `int` care nu întoarce nimic”, a doua și a treia oară

`parcurgerea-vectoriilor.h:99` (`punctul_c`) · `:142` (`punctul_g`) · `:17` (`afisareaElemVect`, de la runda 1)

CUM E ACUM: după ce am reparat erorile de sintaxă, compilatorul dă:

```
pv.h:17:1: warning: non-void function does not return a value
pv.h:99:1: warning: non-void function does not return a value
pv.h:142:1: warning: non-void function does not return a value
```

DE CE: mecanismul e cel de la B4, runda 1 — `int` e o promisiune că funcția întoarce un întreg, iar ajungerea la `}` fără `return` o lasă neîmplinită, cu apelantul citind ce s-a nimerit în registru.

Dar aici e altceva de spus. Amândouă funcțiile noi **doar tipăresc** — `cout << nou`, `cout << p`. N-au ce să întoarcă. Tipul lor corect e `void`, exact ca la `afisareaElemVect`, pe care runda 1 îl semnalase și care a rămas neatins. Trei funcții, aceeași greșeală, și cea mai veche e încă acolo.

Fișa cerea explicit „funcțiile întorc rezultatul, `cout` doar în `main()`”. Dacă `punctul_c` ar fi întors ceva — numărul găsit, sau un `bool` — ar avea sens să fie `int`. Așa cum e scrisă, scrie direct pe ecran, deci `void`.

Cele de la runda 1 care sunt tot deschise, în `while-ex.h`: `:144`, `:190`, `:331`. Cu astea două noi, suntem la **șase locuri** cu aceeași greșeală.

FIX: `void punctul_c(...)`, `void punctul_g(...)`, `void afisareaElemVect(...)`. Sau, mai aproape de fișă: întoarce rezultatul și tipărește-l în `main()`.

🟡 Importante

M7 — Ai rescris două funcții pe care le aveai deja. `isPrim` (`:35`) e `estePrim` din `while-ex.h:248`. `CifraMinima` (`:146`) e `cifraMinima` din `while-ex.h:483`. Amândouă copiile sunt **corecte** — le-am rulat — deci n-ai pierdut nimic pe calitate. Dar ai scris de două ori ce aveai scris o dată, și ai acum două locuri de reparat dacă vreodată apare un bug. Motivul nu e lene: `main.cpp` nu include `while-ex.h`, deci funcțiile vechi **nu-ți sunt accesibile**. B1 de la runda 1 nu e doar „niște cod necompilat” — te obligă să rescrii. Repară includul și dispare și asta.

M8 — `int c[10]` cu indexare de la 1 ține doar 9 cifre. `:80` și `:105`. Indicii valizi sunt `0..9`. Punând prima cifră în `c[1]`, un număr de 10 cifre (orice peste 999 999 999) ajunge să scrie în `c[10]` — în afara vectorului. Fie `int c[11]`, fie indexare de la 0.

M9 — `punctul_g` folosește `nr` înainte de a-l seta. `:121` citește `nr` în condiția buclei. Dacă `while(s > 9)` de la `:109` nu intră nici o dată (numărul are o singură cifră), `nr` n-a fost atins și are valoare nedefinită. Inițializează-l la declarare: `int nr = 0;`.

M1, M2, M3, M4, M5, M6 de la runda 1 sunt toate deschise. M1 (garda de include cu cratimă) acum tot în două fișiere: `parcurgerea-vectoriilor.h:1` dă de fapt macro-ul `PARCURGEREA`, nu `PARCURGEREA_VECTORIILOR_H_INCLUDED` — compilatorul mi-a confirmat-o cu `extra tokens at end of #ifndef directive`.

🟢 Cleanups

C7 — `:146 CifraMinima` — majusculă în mijlocul primului cuvânt. `:66 soll2` cu doi `l`. Sunt nume pe care le vei scrie greșit la apel; `cifraMinima` și `sol2`.

C8 — `:31 //todo:cate nr prime avem in vector`, deasupra funcției care face exact asta. Șterge `todo` -ul când ai rezolvat; altfel peste o lună nu mai știi ce e făcut.

C9 — `:24` și `:69 : int dim` declarat, apoi la apel trimiți literalul (`afisareaElemVect(v, 20)`, `contorPrimeVector(v, 5)`). Două warning-uri `unused variable 'dim'`. E C1 de la runda 1, acum în două locuri.

C10 — `main.cpp:9` `int v[300]` și `cin >> n` fără nicio limită pe `n`. Dacă se citește `n = 500`, bucla scrie dincolo de vector.

C11 — `main.cpp:24`: pe vector vid (`n = 0`) tipărește `9`, valoarea de pornire. Dacă enunțul garantează `n >= 1`, nu e o problemă — dar atunci merită un comentariu care s-o spună.

C12 — `recapitulare.exe` și `main.o` continuă să fie urmărite în git. Commit-ul `fdda59a` a mai cărat 1 MB de binar care **nu corespunde surselor din același commit**. Un `.gitignore` cu `bin/` și `obj/` rezolvă definitiv.

C13 — Mesaj de commit: `finished recap vector hw`. Mai bine: „ex 1c, 1g, 2a, 2c: palindrom, radacina digitala, cifra minima”.

Ce e bine, și merită spus pe nume

CifraMinima e scrisă în forma corectă, și ai nimerit și garda. Rulat:

```
CifraMinima(4738) -> 3    CifraMinima(4038) -> 0
CifraMinima(7)     -> 7    CifraMinima(0)     -> 0
```

Extragi cifra și o folosești **în aceeași iterație**, apoi `n = n / 10` la final. Asta e exact regula pe care runda 1 o ceruse la B2, pentru trei bucle din `while-ex.h` — și ai scris-o corect fără să fi citit runda 1. Plus `if(n == 0) return 0;`, care nu era obligatoriu și pe care majoritatea o uită.

areCifra e corectă pe toate cazurile, inclusiv cel neplăcut.

```
areCifra(12321, 1) -> true    areCifra(4738, 5) -> false
areCifra(0, 0)     -> true    areCifra(0, 5)     -> false
```

`areCifra(0, 0)` e cazul pe care bucla `while(n > 0)` nu-l poate acoperi niciodată, fiindcă nu pornește. Ai pus `if` înaintea ei. `areCifra(0, 5)` arată că `if`-ul e scris cu `&&`, nu doar pe `n == 0` — deci te-ai gândit la amândouă.

isPrim e corectă, cu `i * i <= n`.

```
isPrim(1) -> false    isPrim(2) -> true
isPrim(9) -> false    isPrim(7) -> true
```

`i * i <= n` în loc de `i <= n / 2` e optimizarea corectă, și ai scris-o fără `sqrt`, cum cerea fișa.

Algoritmul din `main()` pentru 2a e corect. Am reparat `cifminn` în `cifmin` și l-am rulat:

```
4 / 4738 91 205 67 -> 0    corect (din 205)
3 / 57 39 68       -> 3    corect (din 39)
1 / 7              -> 7    corect
```

Ai spart problema în două: o funcție care răspunde pentru un număr, și o buclă în `main` care ia minimul peste vector. Fix tiparul cerut. Singurul lucru care-l oprește e un `n` în plus la numele variabilei.

Ai reindentat `sol1` și ai adăugat `parcurgerea-vectoriilor.h` ca `<Unit>` în `.cbp`. Primul e curățenie pe cod deja scris, lucru pe care puțini îl fac.

Before / After

B8 — acolada care închide `sol12`

Acum, `parcurgerea-vectoriilor.h:66-77`:

```
void sol12(){
    int v[100] = {23, 5, 4, 3, 7};
    int dim = 5;
    cout<<  contorPrimeVector(v, 5);

//TEMA
//EX1/sub-C
int punctul_c(int v[], int n)
```

Cum ar trebui:

```

void soll2(){
    int v[100] = {23, 5, 4, 3, 7};
    int dim = 5;
    cout << contorPrimeVector(v, dim);
}

//TEMA
//EX1/sub-C
void punctul_c(int v[], int n)

```

B9 — contorul vs. elementul

Acum:

```

int contorPrimeVector(int v[], int dim)
{
    int ct = 0;
    for(int i = 0; i <= dim; i++)
    {
        if(isPrim(i) == true)
        {
            ct++;
        }
    }
    return ct;
}

```

Cum ar trebui:

```

int contorPrimeVector(int v[], int dim)
{
    int ct = 0;
    for(int i = 0; i < dim; i++)
    {
        if(isPrim(v[i]) == true)
        {
            ct++;
        }
    }
    return ct;
}

```

B12 — rădăcina digitală care se termină

Acum:

```

while(s > 9)
{
    nr = 0;
    for(x = s; x > 0; x = x / 10)
    {
        nr++;
        c[nr] = x % 10;
    }
    s = s + c[j];
}
s = 0;
for(j = 1; j <= nr; j++)
{
    s = s + c[j];
}

```

Cum ar trebui:

```
while(s > 9)
{
    nr = 0;
    for(x = s; x > 0; x = x / 10)
    {
        nr++;
        c[nr] = x % 10;
    }
    int suma = 0;
    for(j = 1; j <= nr; j++)
    {
        suma = suma + c[j];
    }
    s = suma;
}
```

Q&A — răspunde în scris, înainte de runda 3

1. Compilatorul ți-a dat 7 erori, în 5 locuri diferite din două fișiere. Cinci dintre ele au o singură cauză. Care, și cum ți-ai putut da seama din mesaje care anume era cauza și care erau doar urmările ei?
 2. `contorPrimeVector({10, 12, 14}, 3)` întoarce `2`, deși în vector nu e niciun număr prim. Fără să rulezi nimic, există un semn în corpul funcției care spune sigur că răspunsul nu poate fi corect — indiferent de ce numere primește. Care e?
 3. `.exe`-ul din commit-ul tău e mai nou decât cel dinainte, deci ai avut un build reușit. Dar sursa din **același** commit nu compilează. Ce s-a întâmplat între ultimul F9 reușit și `git commit`, și ce pui în `.gitignore` ca repo-ul să nu mai poată conține această contradicție?
-

Runda 1 — 2026-09-24 — sha 2734bb4

Ai scris mult între 17 și 24 septembrie: `while-ex.h` a crescut de la 117 la 564 de linii, plus două fișiere noi. Am compilat tot și am **rulat fiecare funcție** cu exemplele din fișe. Nimic din ce urmează nu e dedus din citit.

Poarta de compilare: ✅ proiectul se compilează (exit 0), cu 5 warning-uri.

🔴 Critice

B1 — `while-ex.h` nu mai e compilat deloc

`main.cpp:1`

CUM E ACUM: `main.cpp` începe cu o singură linie:

```
#include "parcurea-vectoriilor.h"
```

Cele 449 de linii pe care le-ai scris în `while-ex.h` nu mai ajung la compilator. Apeși F9, se face verde, și tu crezi că e în regulă.

DE CE: Code::Blocks compilează doar fișierele `.cpp`. Un `.h` ajunge la compilator **numai** dacă un `.cpp` îl include — direct sau printr-un lanț de include-uri. Faptul că `while-ex.h` apare ca `<Unit>` în `recapitulare.cbp` îl arată doar în arborele proiectului; nu-l compilează.

Până pe 17 septembrie `main.cpp` includea `while-ex.h`, deci codul tău trecea prin compilator. Acum nu mai trece. Dovada: când l-am compilat eu separat, au ieșit **11 warning-uri**, dintre care trei de forma `non-void function does not return a value`. Tu n-ai văzut niciunul, fiindcă fișierul n-a fost atins.

Toate constatările B2–B7 de mai jos sunt în cod pe care compilatorul nu l-a citit de o săptămână.

FIX: ține în `main.cpp` un `#include` pentru fiecare fișier la care lucrezi, și în `main()` un apel la fiecare funcție nouă. Dacă nu vrei să ruleze, măcar include-le: compilatorul tot le verifică.

B2 — Aceeași greșală de buclă, în trei locuri

```
while-ex.h:72 (verificifre) · while-ex.h:425 (cifraMaxima) · while-ex.h:344  
(areToateCifrelePrime)
```

CUM E ACUM: rulat, cu rezultatele tale în stânga și cele corecte în dreapta:

```

cifraMaxima(8123)      -> 3      trebuia 8
cifraMaxima(91)        -> 1      trebuia 9
cifraMaxima(7)         -> -1     trebuia 7
areToateCifrelePrime(42) -> true  trebuia false (4 nu e prim)
areToateCifrelePrime(4) -> true  trebuia false
verifcifre(4738, 4)    -> false  trebuia true (4 e chiar prima cifra)
verifcifre(7, 7)       -> false  trebuia true

```

DE CE: toate trei au aceeași formă — scoți o cifră **înainte** de buclă, iar înăuntru o folosești la început și abia apoi scoți următoarea:

```

int uc = n % 10;
n = n / 10;
while(n > 0)
{
    if(uc > maxx) { maxx = uc; }
    uc = n % 10;
    n = n / 10;
}

```

Urmărește ultimul pas pentru `8123`. Ajungi la `uc = 1`, `n = 8`. Condiția `8 > 0` e adevărată, deci intri: compari `1`, apoi faci `uc = 8`, `n = 0`. Te întorci la condiție: `0 > 0` e falsă, ieși. **Cifra 8 a fost scoasă, dar n-a mai apucat să fie comparată.**

Bucula ta compară `nrCifre - 1` cifre, nu `nrCifre`. Cifra care rămâne mereu neverificată e ultima extrasă — adică prima din stânga numărului. De asta la un număr de o singură cifră bucla nu pornește deloc și `cifraMaxima(7)` dă `-1`: valoarea de pornire, neatinsă.

Că știi forma corectă se vede la 60 de linii mai jos. `cifraMinima`, pe care ai scris-o tu, face extragerea și folosirea **în aceeași iterație**:

```

while(n > 0)
{
    int uc = n % 10;
    if(uc < minn) { minn = uc; }
    n = n / 10;
}

```

Aici nicio cifră nu scapă: cât timp mai există o cifră, bucla intră și o folosește. Diferența dintre cele două forme e tot ce desparte `cifraMinima(7) -> 7` de `cifraMaxima(7) -> -1`.

Colateralul se vede cel mai bine în EX 1 și EX 2. Sunt același algoritm, scris de tine de două ori, corect amândouă. Doar ajutorul diferă:

```
celMaiMareNumarCORECT(4738) -> 8729      trebuia 8743      (cheama cifraMaxima)
celMaiMareNumarCORECT(7)      -> -1        trebuia 7
celMaiMicNumarCORECT(4738)    -> 3478      corect          (cheama cifraMinima)
celMaiMicNumarCORECT(4038)    -> 348       corect
celMaiMicNumarCORECT(1203)    -> 123       corect
```

`celMaiMareNumarCORECT` nu are nimic greșit în el. Pierde pentru că se sprijină pe o funcție care minte. Asta e și motivul pentru care merită reparat `cifraMaxima`, nu `celMaiMareNumarCORECT`.

FIX: rescrie cele trei bucle după modelul lui `cifraMinima` — extragi și folosești în aceeași iterație, `n = n / 10` la final.

B3 — Două funcții se blochează la infinit

```
while-ex.h:126 ( celMaiMareNumar ) · while-ex.h:171 ( celMaiMareNumar2 )
```

CUM E ACUM: `celMaiMareNumar(4738)` nu întoarce niciodată. Am oprit-o după 3 secunde de procesor. La fel `celMaiMareNumar2(4738)`.

DE CE: bucla exterioară e `while(n > 0)`, dar în corpul ei `n` nu se schimbă niciodată. La `celMaiMareNumar` lucrezi pe `aux = n / 10` — se micșorează `aux`, nu `n`. La `celMaiMareNumar2` nici măcar nu-l atingi.

O buclă `while` nu are un pas automat, cum are `for`. Condiția e recită la fiecare tur, dar din aceleași variabile. Dacă niciuna dintre variabilele din condiție nu e modificată în corp, condiția va da la infinit același răspuns. `4738 > 0` a fost adevărat la primul tur și rămâne adevărat.

Nu le-ai văzut fiindcă nu le-a apelat nimeni din `main()` — vezi B1.

FIX: ambele sunt încercări abandonate; `celMaiMareNumarCORECT` e varianta care merge. Șterge-le, sau pune-le `n = n / 10` ca să se termine. Dar nu le lăsa așa: prima dată când le apelezi, programul îngheață fără niciun mesaj.

B4 — Patru funcții `int` care nu întorc nimic

`while-ex.h:144` · `while-ex.h:190` · `while-ex.h:331` · `parcurgerea-vectoriilor.h:17`

CUM E ACUM: compilatorul spune, de trei ori în `while-ex.h` și o dată în fișierul de vectori:

```
warning: non-void function does not return a value [-Wreturn-type]
```

DE CE: ai scris `int numeFuncție(...)`, adică ai promis că funcția întoarce un întreg. Când execuția ajunge la `}` fără să treacă printr-un `return`, promisiunea rămâne neîndeplinită. În C++ asta nu e eroare, e **undefined behavior**: apelantul citește ce se nimerește în registrul în care ar fi trebuit pus rezultatul. Poate fi 0, poate fi o valoare de la apelul dinainte. Același program poate da alt răspuns în Debug și în Release.

La `cmmdc` (`:331`) e mai subtil: ai `return` în ambele `if`-uri, dar compilatorul nu poate demonstra că măcar unul se execută mereu, așa că vede o cale prin care se ajunge la `}`. De asta warning-ul zice `in all control paths`.

La `afisareaElemVect` problema e alta: funcția doar afișează, deci **nu are ce să întoarcă**. Tipul ei ar trebui să fie `void`.

FIX: `void` unde nu ai ce întoarce; `return` pe toate căile unde ai.

B5 — `cmmdc(0, 0)` face împărțire la zero

`while-ex.h:312`

CUM E ACUM: rulat sub sanitizer:

```
while-ex.h:312:18: runtime error: division by zero
```

DE CE: când `a` și `b` sunt amândoi 0, intri pe prima ramură cu `i = a`, adică `i = 0`. Condiția `i <= a` e `0 <= 0`, adevărată, deci se execută `a % i` — adică `0 % 0`.

Împărțirea întreagă la zero nu e ca cea cu numere reale: nu dă „infiniț”, ci oprește programul. Pe Windows cu MinGW asta înseamnă crash pe loc, fără mesaj util.

FIX: tratează cazul înainte de buclă — `cmmdc(0, 0)` e 0, iar `cmmdc(x, 0)` e `x`.

B6 — `eliminareCifra` scoate toate aparițiile, nu una

`while-ex.h:448`

CUM E ACUM:

```
eliminareCifra(44, 4)  -> 0      trebuia 4
eliminareCifra(121, 1) -> 2      trebuia 21
celMaiMicNumarCORECT(4400) -> 400   trebuia 44
```

DE CE: condiția `if(cifra != cifraEliminata)` se aplică la **fiecare** cifră pe care o parcurgi. Dacă cifra căutată apare de trei ori, toate trei sunt sărite. Funcția pe care ai scris-o răspunde la „scoate cifra 4” cu „scoate toți de 4”.

Asta strică EX 1 și EX 2 exact pe numerele cu cifre repetate. Algoritmul tău zice: ia cifra cea mai mică, scoate-o din număr, pune-o pe poziția curentă, repetă. Când scoți din `4400` cifra minimă `0`, el scoate **amândoi** de 0, și ai pierdut o cifră din numărul pe care trebuia să-l reconstruiești.

FIX: ai nevoie de un contor care să oprească ștergerea după prima apariție — un `bool sters = false`, pus pe `true` când ai sărit o cifră, și de acolo înainte copiezi tot.

B7 — `celMaiMicNrCuSumaCifrelor` greșește pe sume mari

`while-ex.h:386`

CUM E ACUM:

```
celMaiMicNrCuSumaCifrelor(5)  -> 5      corect
celMaiMicNrCuSumaCifrelor(10) -> 19     corect
celMaiMicNrCuSumaCifrelor(20) -> 99     trebuia 299
```

DE CE: ideea ta e bună — umpli de la dreapta cu cifre cât mai mari, ca numărul să aibă cât mai puține cifre. Dar bucla interioară `while(i <= s - a)` adaugă `9` cât timp mai încapе un `9` **întreg**, și se

oprește când restul e mai mic decât `i`. La `s = 20` pui doi de 9, ajungi la `a = 99` cu suma cifrelor 18, mai ai nevoie de 2 — iar condiția `9 <= 20 - 99` e falsă, deci bucla exterioară se termină cu `a = 99` în loc să mai pună un `2` în față.

Problema e că testezi `i <= s - a`, unde `a` e **numărul construit**, nu suma cifrelor lui. După ce `a` devine 99, `s - a` e negativ și toate comparațiile devin false. Îți trebuie două variabile separate: numărul care se construiește și cât din sumă ai acoperit deja.

FIX: ține un `sumaRamasa = s` și scade din ea cifra pusă, în loc să compari cu numărul construit.

● Importante

M1 — Garda de include cu cratimă, acum în două fișiere. `while-ex.h:1` și `parcurgerea-vectoriilor.h:1`. Cratima nu e caracter permis într-un nume de macro, deci preprocesorul citește doar `WHILE`, respectiv `PARCURGEREA`. Merge din întâmplare; la al doilea fișier cu același prim cuvânt s-ar ciocni. Corect: `WHILE_EX_H_INCLUDED`, `PARCURGEREA_VECTORIILOR_H_INCLUDED`.

M2 — `(nou / p) % p` nu calculează ce crezi. `while-ex.h:156` și `:203`. Vrei ultima cifră a lui `nou / p`, deci `% 10`. Cu `% p` compari cu altceva la fiecare tur, fiindcă `p` crește. Rulat: `celMaiMareNumar1(4738) -> 705`, `celMaiMicNumar(4738) -> 8374`.

M3 — `pow` folosit fără `#include <cmath>`. `while-ex.h:475` și `:550`. Merge doar fiindcă `<iostream>` îl aduce indirect, și nu e garantat. În plus `pow` întoarce `double`: pentru exponenți mari `cifraM * pow(10, ctCif)` poate ieși cu o unitate mai mic după conversia la `int`. Pentru puteri ale lui 10 e mai sigur un `p = p * 10` într-o buclă.

M4 — `cmmmc` poate depăși. `while-ex.h:337`: `a * b` se calculează **înainte** de împărțire. Pentru `a = b = 100000` produsul nu mai încapă în `int`. Scris ca `a / cmmdc(a, b) * b` împarți întâi, deci rezultatul intermediar rămâne mic.

M5 — `fisa2.h` e gol și comis. Doar garda și `#include <iostream>`. E și listat ca `<Unit>` în `.cbp`.

M6 — Definiții complete de funcții în `.h`. Merge cât timp ai un singur `.cpp`. La al doilea, linker-ul se oprește cu `multiple definition` pentru fiecare funcție. Garda `#ifndef` apără o unitate de compilare, nu două.

● Cleanups

- C1** — `parcurgerea-vectoriilor.h:24` : `int dim = 20;` declarat, dar la apel trimiți literalul `20`. Dacă schimbi `dim`, apelul rămâne pe 20. Vectorul are de fapt **24 de valori**, deci ultimele 4 nu se afișează.
- C2** — `parcurgerea-vectoriilor.h:12` : `int n;` nefolosit.
- C3** — `while-ex.h:544` : `isZero` setat și niciodată citit.
- C4** — `bin/Debug/recapitulare.exe` și `obj/Debug/main.o` sunt urmărite în git. Se rescriu la fiecare build, deci fiecare commit cară ~1 MB. Repo-ul a crescut de la 592 KB la 920 KB în trei commit-uri. Un `.gitignore` acum e ieftin; peste 50 de commit-uri, nu mai e.
- C5** — `while-ex.h:8` : `//todo: cate cifre are n`, deasupra unei funcții care numără cifre pare și impare. `main.cpp:2` : `//EX 12: ??`.
- C6** — Mesaje de commit: `tema`, `finished hw`, `commit 24.09`. Peste o lună nu-ți mai spun nimic. Scrie ce ai făcut: „ex 4-7: divizori, prim, perfect”.
-

Ce e bine, și merită spus pe nume

EX 4-7 sunt corecte, verificate prin rulare, inclusiv pe capete:

```
nrDivizori(12) -> 6      nrDivizori(7) -> 2      nrDivizori(1) -> 1
estePrim(7) -> true      estePrim(2) -> true
estePrim(9) -> false     estePrim(1) -> false
sumaDivizoriloProprii(12) -> 16      (7) -> 1      (1) -> 0
estePerfect(6) -> true  estePerfect(28) -> true  estePerfect(12) -> false
```

`estePrim(1) -> false` și `estePrim(2) -> true` sunt exact cele două capete pe care le ratează aproape toată lumea. Le-ai nimerit pe amândouă.

`cmmdc` și `cmmmc` dau corect pe toate exemplele din fișă — 6, 25, 1, 12, 91. Singura fisură e `cmmdc(0, 0)` de la B5.

Ai spart problema grea în ajutoare, fără să ți-o ceară nimeni. `celMaiMareNumarCORECT` e construit din `numarDeCifre`, `cifraMaxima` și `eliminareCifra`, fiecare scrisă separat și testabilă separat. Fix asta cerea fișa prin „nu rescrie ce ai deja”, și e motivul pentru care B2 se repară într-un singur loc, nu în patru.

Continui să trasezi pe hârtie. Tabelele din :442 , :499 , :510 și :421 sunt, la nivelul tău, cel mai bun obicei din tot repo-ul. Tabelul de la :510 pentru `inserareCifre` chiar prezice corect rezultatul.

Marchezi cinstit ce nu ți-a ieșit — `// VARIANTA 1 - ?` , `//EX 3: ??` , codul comentat păstrat în loc de șters. Nu te prefaci că e gata. E exact ce trebuie făcut.

Before / After

B2 — bucla care pierde ultima cifră

Acum, în `cifraMaxima` :

```
int uc = n % 10;
n = n / 10;
while(n > 0)
{
    if(uc > maxx) { maxx = uc; }
    uc = n % 10;
    n = n / 10;
}
return maxx;
```

Cum ar trebui — forma pe care ai scris-o deja în `cifraMinima` :

```
int maxx = -1;
while(n > 0)
{
    int uc = n % 10;
    if(uc > maxx) { maxx = uc; }
    n = n / 10;
}
return maxx;
```

B4 — funcție care doar afișează

Acum:

```
int afisareaElemVect(int v[], int dim)
{
    int n;
    for(int i = 0; i < dim; i++)
    {
        cout << v[i] << " ";
    }
}
```

Cum ar trebui:

```
void afisareaElemVect(int v[], int dim)
{
    for(int i = 0; i < dim; i++)
    {
        cout << v[i] << " ";
    }
}
```

B6 — ștergerea unei singure apariții

Acum, în `eliminareCifra` :

```
if(cifra != cifraEliminata)
{
    nou = cifra * p + nou;
    p = p * 10;
}
```

Cum ar trebui:

```
if(cifra != cifraEliminata || sters == true)
{
    nou = cifra * p + nou;
    p = p * 10;
}
else
{
    sters = true;
}
```

cu `bool sters = false;` declarat înainte de buclă.

Q&A — răspunde în scris, înainte de runda 2

1. Ai apăsat F9 și s-a compilat fără erori. Totuși `cifraMaxima(8123)` întoarce 3. Cum se poate ca un program care se compilează curat să conțină o funcție care dă răspuns greșit — și de ce în cazul tău compilatorul nici măcar n-a apucat să se uite la ea?
2. `celMaiMicNumarCORECT(4738)` dă 3478, corect. `celMaiMareNumarCORECT(4738)` dă 8729, greșit. Cele două funcții sunt scrise la fel. Unde e diferența, și de ce nu în ele trebuie căutată?
3. Ai trei bucle cu aceeași greșeală (`verifcifre` , `cifraMaxima` , `areToateCifrelePrime`) și una corectă (`cifraMinima`). Scrie în cuvintele tale regula care le desparte — o singură frază, care să funcționeze ca test pentru orice buclă pe cifre pe care o vei scrie de acum înainte.

